

5.5 - Pipeline como código: infraestrutura, reprodutibilidade e manutenção

O pipeline também é código, com o mesmo rigor da aplicação

O pipeline também é código, e exige o mesmo rigor

Tratar pipeline como configuração ad-hoc é abrir buraco de segurança

- **Pipeline definido em interface web não tem histórico de mudanças**
 - Mudança aprovada por ninguém pode entrar em produção em minutos
- **Definição em código permite versionamento no mesmo repositório da aplicação**
 - Cada mudança vira commit, cada commit é revisado em pull request
- **Pipeline-as-code garante execução idêntica hoje e em um ano**
 - Reprodutibilidade é base para auditoria e investigação de incidentes
- **Permite testar mudanças em branches antes de aplicar na main**
 - PR de pipeline roda contra si mesmo, validando que ainda funciona

Princípios do pipeline-as-code

Quatro práticas que separam disciplina de bagunça

- **Definição versionada no repositório, próxima ao código que ela orquestra**
 - Arquivos como `.github/workflows` ou `.gitlab-ci.yml`, lado a lado com `src`
- **Pull request obrigatório para qualquer mudança na pipeline**
 - Revisão por par, especialmente para mudança que afeta gates de segurança
- **Segredos nunca no arquivo, sempre via secret manager do orquestrador**
 - Segredos comitados em YAML são prontamente expostos em histórico
- **Pipeline executa testes da própria pipeline em branches**
 - Mudança quebrada em main paralisa todos os deploys, não dá para descobrir em produção

A 'otimização' silenciosa que custou caro à LunaLog

- **Pipeline definido em YAML editável diretamente pela interface do GitHub Actions**
 - Sem histórico de mudanças, sem revisão por par, sem testes da própria pipeline

LUNALOG

Em uma tarde de fim de sprint, um desenvolvedor da LunaLog estava com pressa. O pipeline rodava em onze minutos, o stage de testes de integração consumia sete. Ele abriu a interface web do GitHub Actions, comentou o stage de integração e salvou. O pipeline passou a rodar em quatro minutos, todo o time achou que era ganho de produtividade. Duas semanas depois, um bug que os testes de integração teriam capturado chegou à produção, indisponibilizando o serviço de cotação por seis horas. Ninguém sabia quem havia removido os testes, nem quando, nem por quê. Pipeline como código teria exigido pull request, revisão e aprovação para essa mudança. O incidente daquele dia custou um cliente.

IaC como pré-requisito de pipeline reprodutível

- Pipeline reprodutível exige infraestrutura também definida em código
- Servidor configurado manualmente é caixa preta dentro do pipeline

Sem Infrastructure as Code, pipeline-as-code é meia adoção. O artefato pode ser reproduzível, mas o ambiente onde ele roda continua sendo um floco de neve. Terraform, Pulumi, CDK ou equivalente precisam estar no mesmo nível de disciplina que o pipeline. Caso contrário, qualquer recriação de ambiente em incidente vira arqueologia.

- Pipeline-as-code e IaC são duas faces da mesma maturidade operacional



Pipeline como código no lugar, segredos protegidos, mudanças revisadas. Agora chegamos à estratégia de deploy em si, o momento de maior risco em qualquer sistema em produção. Como trocar a versão de um serviço de tracking com transportadoras operando vinte e quatro horas por dia, sem que nenhuma perceba a transição?

Blue/green e rolling deployment, troca de versão com controle